Dialogue Manager

The `DialogueManager` is the primary component that you will need to understand when working with the Dialogue System. The `DialogueManager` is responsible for managing the state of a dialogue interaction at runtime and providing Dialogue information.

The `DialogueManager` does not provide any UI or visualization of dialogue. It is intended that the frontend/UI for dialogue be created by the developer to best suit their use-case. The API of the `DialogueManager` has been designed to make this as seemless as possible.

A `DialogueManager` component can only have one active dialogue at a time. If you require multiple ongoing interactions, such as for allowing conversations to resume when leaving/returning to an NPC, then a separate instance of `DialogueManager` should be used for each.

Beginning New Dialogue

```
void BeginDialogue(DialogueAsset dialogueAsset, string startName="")
```

To tell a `DialogueManager` to begin a new dialogue you must call `DialogueManager.BeginDialogue`. The `dialogueAsset` is the `DialogueGraph` which you wish to begin. The `startName` is an optional string field that is to be used specify the start point to use. The `startName` field is primarily used for graphs with multiple start points, as any graph with one start point is advised to leave its name blank.

> This method invokes DialogueManager.StartedDialogue UnityEvent.

> Calling this method on a DialogueManager that has already begun will do nothing and log an error to the console.

Getting the Next Dialogue

```
DialogueInfo AdvanceDialogue(AdvanceContext context)
DialogueInfo AdvanceDialogue()
```

Once a dialogue interaction has begun you must call the `DialogueManager.AdvanceDialogue` method in order to retrieve the next output for the interaction. These methods each return an instance of `DialogueInfo`, which contain all the relevant information for the next dialogue. If the `DialogueInfo` is null, this indicates that the end of the dialogue has been reached.

Calling this method prompts the `DialogueManager` to walk through the `DialogueGraph` and will perform other operations such as calling the `DSEvents` of `EventNode`s, Evaluating conditions of Redirect Options,

and setting `Variable` values for `SetVariableNode`s until it reaches a node with an output. All these operations will be performed prior to returning and in the order of the graph.

The `AdvanceContext` parameter to specify the response of the player for choice dialogue. For regular dialogue it has no effect and can be optionally excluded. You can also define your own child class of AdvanceContext to provide to this method. `OptionType`s will receive an instance of this AdvanceContext instance when evaluating their condition, so custom `AdvanceContext` types can be used alongside custom `OptionsType`s to express more complex behaviors.

> This method invokes DialogueManager.AdvancedDialogue UnityEvent.

> Calling this method before beginning a dialogue will return null and log an error to the console.

Refreshing Dialogue

```
DialogueInfo RefreshDialogue(AdvanceContext context)
DialogueInfo RefreshDialogue()
```

Refreshing the dialogue can be useful if conditions or variables change over time and you would like your frontend to reflect that. Refreshing dialogue does not restart or walk backwards through the graph, it only re-returns the output of the current dialogue after re-applying `Variables` and re-evaluating available responses.

The `AdvanceContext` instance from the previous `AdvanceDialogue` or previous `RefreshDialogue` call can be reused if a new context is not provided.

> Calling this method before beginning a dialogue will return null and log an error to the console.
>
> Calling this method the first AdvanceDialogue has been called will return null and log an error to the console.
>
> > If either error is encountered the stored AdvanceContext will not be updated

Ending a Dialogue

```
void EndDialogue()
```

This method is used if you wish to end the current dilogue prematurely. If the dialogue ends naturally this method is called before `AdvanceDialogue` returns and doesn't need to be called manually. Remember that `AdvanceDialogue` returning null means that the dialogue has ended.

> This method invokes DialogueManager.EndedDialogue UnityEvent.

> Calling this method before beginning a dialogue will return null and log an error to the console.

Variables

The `DialogueManager` stores `Variable`s using a very similar API to Unity's Animator class. You can store variables of types Bool, Int, Float, and String. Each `Variable` has a string identifier. `Variable` values are local to each DialogueManager and are not shared between instances.

To Assign variables you use methods in the for of `DialogueManager`.SetXXX. Setting a variable will assign the variable of that name to a new value, even if it is of a different type than the old value. `Variables` can also be set witin the `DialogueManager`'s Inspector to assign initial values.

```
DialogueManager.SetString(string name, string value)
DialogueManager.SetBool(string name, bool value)
DialogueManager.SetFloat(string name, float value)
DialogueManager.SetInt(string name, int value)
```

Retriving a value uses a simialar set of mathods in the form `DialogueManager`.GetXXX. Setting a `Variable` will return the currently assigned value. Attempting to retrieve a value that isn't assigned or is of the wrong type will return the default value for the respective type and log a warning to the console. If the currently running dialogue has a default value assigned and of the correct type then that value will be returned instead

```
DialogueManager.GetString(string name)
DialogueManager.GetBool(string name)
DialogueManager.GetFloat(string name)
DialogueManager.GetInt(string name)
```

Several methods also exists for setting variables using the `Variable` type. The `Variable` type is used internally to store variables and provides more flexibility in retrieving variables of unknown type.

```
bool TryGetVariable(string name, out Variable variable)
void SetVariable(string name, Variable variable)
```

Dialogue Parameters

While the `DialogueInfo` stores all information and parameters of individual dialogue, `DialogueParameters` store information for the entire interaction. The current parameters are based on the start point that was

used to begin the interaction. The `DialogueManager` has multiple methods pertaining to these parameters.

```
Type GetDialogueParamsType()
T GetDialogueParams<T>() where T : DialogueParameters
```

`DialogueManager.GetDialogueParamsType` returns the runtime type of the current parameters. This type will always be derived from `DialogueParameters`. The Generic Getter method can then be used to get the parameter class as a type or as the base class. If the parameters are null or of the wrong type null will be returned.

Dialogue Info

`DialogueInfo` contains all the information pertaining to a specific dialogue output. It is returned by the `DialogueManager` when advancing or refreshing dialogue via `DialogueManager.AdvanceDialogue` or `DialogueManager.RefreshDialogue`. If the returned DialogueInfo is null then the dialogue has ended.

There are several public fields of the dialoguInfo. The first is the `dialogueType`, which is an enumerator representing what type of dialogue output this is:

- `DialogueType.Basic` - Regular text dialogue
- `DialogueType.Choice` - Choice dialogue containing response info
- `DialogueType.Stall` - Encountered Event with stall, containing no other info

Text

The `DialogueInfo.text` field contains the string output of the dialogue to be displayed as long as the type is either `DialogueType.Basic` or `DialogueType.Choice`. Text inside of brackets `{}` will be replaced with the current value of a variable. For instance, if the `Variable` 'myVar' were set to 5, "{myVar}" would be replaced by "5".

ResponseInfo

The `DialogueInfo.responses` is a List of `ResponseInfo` instances. These responses contain the info for each individual response. The field will be null unless the type is `DialogueType.Choice`. `ResponseInfo` has a `ResponseInfo.text` field which contains the string for that response. Brackets are also used to embed variable values in the same way as the `DialogueInfo.text`.

The number of responses available is equal to the length of the list. When advancing the dialogue from a `DialogueType.Choice` output, the next `AdvanceContext` should have its `AdvanceContext.choice` field set to the response selected by the user. The choice should be the response's index in the `DialogueInfo.responses` list. If `AdvanceContext.choice` is set to -1, then the 'Default' output of the ChoiceNode will be used instead of any of the options.

Paramaters

If the `DialogueInfo` has any parameters assigned they can be accessed using the following methods:

```
// Methods for Text parameters
Type DialogueInfo.GetBaseParamsType()
T DialogueInfo.GetTextParams<T>() where T : TextParameters

// Methods for Choice parameters
```

```
Type GetChoiceParamsType()
T GetChoiceParams<T>() where T : ChoiceParameters
```

These methods will return null if there is not a corresponding parameter type assigned. Getting the type of the parameter is useful for checking whether the parameter type is present and, if so, what type it is. This can help make the code a bit more clean compared to retrieving the parameters as the base class and attempting to cast to supported derived types.

`ResponseInfo` also has corresponding methods for getting each response's response parameters:

```
// Methods for Response parameters
Type ResponseInfo.GetResponseParamsType()
T ResponseInfo.GetResponseParams<T>() where T : ResponseParameters
```

Dialogue Graphs

Dialogue Graphs are how dialogue interactions are designed and editted. The tools for this are made with Unity's (relatively) new Graph Toolkit. The Graph Toolkit was added in Unity 6.2 as an experimental package, but was added as a built-in package in Unity 6.4. If you want more detailed information about using the Graph editor you can view Unity's documentation

Creating a Dialogue Graph

A new dialogue Graph can be created through the Create menu under `Create -> Dialogue System -> Dialogue Graph`. This will create a new asset which you can name appropriately. Dialogue Graphs are distinguished by the `.dialogue` asset extension. To open the new graph you can double click the newly created asset, which should open a new window in your Unity Editor.

Every Dialogue interaction must begin with a `StartNode`. To create a new node you can right click anywhere in the graph window and select `Add Node`. This will show a list of all available nodes to be added. You will typically begin by adding a `StartNode`. The `StartNode` will have an option for the name of the start point. If your interaction will only have one start pont then you are advised to leave this as an empty string. If you have an `DialogueParameters` defined they will also be added as an option to your `StartNode`s.

Nodes can be connected like any other graph tool, by dragging from a node' port. The connections between nodes represent the control flow of the dialogue. Dialogue Graphs can be read from left to right, two connected nodes represents a possible path for the dialogue to take. Some nodes represent more complex behaviors than others. You should node that the output ports of a node can be connected to multiple other nodes, but this shouldn;t be done. All but one of those connections will be ignored.

The other main nodes are `TextNode`s, `ChoiceNode`s, and `OptionNode`s. Text and Choice nodes represent that two main types of dialogue output. `TextNode`s have only a text output, while `ChoiceNode`s have a text output and provide a decision for the user to make. `OptionNode`s can be attached to `ChoiceNode`s and represent each of the available responses. Connecting the OptionNodes output allows you to choose what will happen is that specific response is selected. The `ChoiceNode` also has a 'Default' output for scenerios where no option was selected.

Creating new variables

Dialogue Graphs also have their own `Variables`. The Dialogue System is configured to handle variables very similarly to the Unity Animator. `Variables` can be added to a graph by opening the blackboard in the top left corner and selecting the '+' button. You can then configure the `Variable`'s name and assign its default value. `Variables` have multiple uses within a Graph. Firstly, Any text inside a set of curly brackets will be replaced by the value of the variable of that name (i.e. {VarName} -> value of 'VarName'). Certain `OptionType`s also use the value of variables.

Redirect Nodes

Redirect nodes are nodes which do not produce any output, but can be used to affect the control flow of the graph. `OptionNode`s can be added to the redirect nodes in the same way as the `ChoiceNode`. The option that is selected, rather than using user input, is chosen by evaluaing the `OptionType` of each `OptionNode`. How these `OptionType`s are evaluated depands on the type of redirect node. `OptionNode`s that are on a redirect node will not have the text field that they have when on `ChoiceNode`s.

Sequential Redirect Node

The `SequentialRedirectNode` evaluates the condition of each `OptionNode`'s `OptionType` in the order that they are listed. It will iterate through until the first option whose condition evaluates to true, and will select that option. If no Options are present or none evaluate to true, then the 'Default' output will be used.

Random Redirect Node

The `RandomRedirectNode` evaluates the condition of all `OptionNode`'s `OptionType`. Any `OptionNode` that is placed on a `RandomRedirectNode` will have an additional 'weight' paraeter. All of the options which evaluate to true will then be chosen at random via a weighted random selection using the Options' weights. If no Options are present or none evaluate to true, then the 'Default' output will be used.

Set Variable Node

The `SetVariableNode` is another node which doesn't produce any output for the dialogue. It is used to change the value of a `Variable` at runtime. This node only affects the value associated with a `Variable` for the `DialogueManager` that is running the dialogue. The graph's assigned default value for the `Variable` will remain unchanged. The default values for variables are static and can only be changed manually within the Graph editor.

> The variable which the node sets is defined using the name of the variable. If a name not defined within the graph is used then a warning will be present on the node.

Custom Manager

The Dialogue System is designed to be as flexible as possible to support a wide range of use-cases. This is primary achieved through the use of custom parameters. There are several different types of parameters associated with different aspects of dialogue.

A given node can only hold one of each type of parameter which it supports. If multiple of a given type are defined then you will be given the option to select which a node should hold. The nodes which support custom parameters are as follows:

- `StartNode` - Dialogue Parameters
- `TextNode` - Text Parameters
- `ChoiceNode` - Text Parameters, Choice Parameters
- `OptionNode` - OptionType, Response Parameters (On Choice Only)

> All Custom parameter classes should be defined with the `[Serializeable]` Attribute. Additionally, if you have multiple of a type of parameter defined, you can use the `[TypeOption]` Attribute to define its name and order within the dropdown.

Dialogue Parameters

Dialogue Parameters are defined by creating a class derived from `DialogueParameters`. Dialogue parameters are parameters that are associated with the entire dialogue interaction as a whole. Havin at least one child class of `DialogueParameters` will make a 'Dialogue' tab available on all `StartNode`s

Dialogue parameters might often include fields about interaction types, UI details, background music, or any other information your dialogue UI/frontend can use that would be redundant to specify on many different nodes.

```
[Serializable, TypeOption("Sample")]
public class MyDialogueParameters : DialogueParameters
{
    [SerializeField] public AudioClip bgm;
}
```

Text Parameters

Text Parameters are defined by creating a class derived from `TextParameters`. Text parameters are parameters that are associated with any dialogue element with text to be displayed, which includes both Regular and Choice dialogue. Having at least one child class of `TextParameters` will make a 'Text' tab available on all appropriate nodes.

Text parameters might often include fields about who is speaking, color of the text, font size, audio, text scroll speed, or any other information your dialogue UI/frontend might use in displaying your dialogue.

```
[Serializable, TypeOption("Sample")]
public class MyTextParameters : TextParameters
{
    [SerializeField] public string speakerName;
}
```

Choice Parameters

Choice Parameters are defined by creating a class derived from `ChoiceParameters`. Choice parameters are parameters that are associated with choice dialogue. Having at least one child class of `ChoiceParameters` will make a 'Choice' tab available on all Choice Dialogue Nodes.

Choice parameters might often include fields about time limits, arrangement of responses, or any other information your dialogue UI.frontend might use in displaying choice dialogue.

```
[Serializable, TypeOption("Sample")]
public class MyChoiceParameters : ChoiceParameters
{
    [SerializeField] public bool isTimed;
    [SerializeField, Min(0)] public float timeLimit;
}
```

Response Parameters

Response parameters are defined by creating a class derived from `ResponseParameters`. Response parameters are parameters that are associated which the response options of a `ChoiceNode`. Having at lease one child class of `ResponseParameters` will make a 'Response' tab available on all `OptionNodes` that are attached to a `ChoiceNode`.

Response parameters might often include fields about requirements for selecting a response, indicators about the effects of responses, text color, font size, hover audio, or any other information your dialogue UI/frontend might use in displaying your choice responses.

```
[Serializable, TypeOption("Sample")]
public class MyResponseParameters : ResponseParameters
{
    [SerializeField] public Sprite indicator;
}
```

Option Type

While they function differently from the other parameter variants, option types are defined and are selected within the graph in much the same way. Option types are defined by creating a class derived from `OptionType`. Option types serve the purpose of defining a condition associated with a `OptionNode`. The exact purpose of ths condition varies by what node the `OptionNode` is attached to.

1. `ChoiceNode` - The OptionType controls whether the response will be available. Unavailable responses are not returned and the frontend is not made aware of them. If you'd like to have a response visible but unable to be selected, this should be achieved using Response parameters and a custom frontend impkenentation.
2. `SequentialRedirectNode` - The `OptionNode`s attached to this node are each evaluated based on their `OptionType`. The `DialogueManager` will traverse through the first node whose condition passes, or through the 'Default' port if no condition passes.
3. `RandomRedirectNode` - Functions very similarly to `SequentialRedirectNode`, but all OptionNodes always have their `OptionType` evaluated. Of all passing conditions, a weighted random option will be selected for the `DialogueManager`. Again, if no conditions pass the output of the 'Default' port will be used instead.

Unlike the other parameter Types, the `OptionType` class does require a function implementation. The `OptionType.EvaluateCondition` is the method used by the aformentioned nodes to determine if the OptionNode's condition passed, in which case this method should return true. The `AdvanceContext` is the same one provided to the call to DialogueManager.AdvanceDialogue which triggered the traversal. The `IVariableContext` provides an interface to access the variables currently held by the `DialogueManager`.

Additionally, if the `OptionType` you are defining uses fields that represent the name of a variable, you should configure your `OptionType` so that a warning can be shown if the variable name isn't defined within the graph. This can be done by overwriting `the CheckVariables` property, which expects a `IEnumerable<string>`

```
[TypeOption("Variable Equals")]
public class VariableEqualsOption : OptionType
{
    [Tooltip("Name of the variable to be compared")]
    [SerializeField] private string variableName;

    [Tooltip("value to compare the variable with")]
    [SerializeField] private Variable value;

    public override bool EvaluateCondition(AdvanceContext advanceContext,
IVariableContext variables)
    {
        if (variables.TryGetVariable(variableName, out var variable))
```

```
            return variable == value;
        return false;
    }

    public override IEnumerable<string> CheckVariables => new List<string> { variableName };
}
```

There are several OptionTypes which are provided by default, as they are extremely commonly used or needed:

1. Basic Option - Always returns true, usually used on ChoiceNode responses that should always appear.
2. Variable Equals - Returns true if the variable specified by name matches a specified value.
3. Random Chance - Returns true with specified probability

Dialogue Events

Invoking events from within dialogue is accomplished through the use of the `EventNode` along with `DSEvent` objects.

Creating an Event Object

Creating a new event requires creating a new `DSEvent` object. This can be done through the create menu by navigating to `Create -> Dialogue System -> Events` and selecting the type of event that you wish to create. While custom event types can be created, there are several types of events that are included by default due to being frequently needed. Those types are:

- `DSEvent` - Non-generic type used for basic event that require no data.
- `DSEvent (string)` - Defined as `DSEventString : DSEvent<string>`
- `DSEvent (bool)` - Defined as `DSEventBool : DSEvent<bool>`
- `DSEvent (float)` - Defined as `DSEventFloat : DSEvent<float>`
- `DSEvent (int)` - Defined as `DSEventInt : DSEvent<int>`
- `DSEvent (AudioClip)` - Defined as `DSEventAudioClip : DSEvent<AudioClip>`

It is also important to note, that when storing references to a specific type of event that you use the closed generic form. For example, use `DSEventInt` rather than `DSEvent<int>`. Unity doesn't handle generic SpriptableObject fields well, and will show all events in the dropdown rather than just those of the correct type. Using the closed generic prevents this issue.

Events inside a Graph

Using an event within a `DialogueGraph` requires creating a `EventNode`. The `EventNode` can be placed anywhere within your dialogue sequence and will Invoke the assigned event when traversed by a `DialogueManager`. The `EventNode` supports assigning both typed and untyped events. In the case that a typed event is used, the `EventNode` will provide a field to enter the value you'd like the event to be called with for that specific invocation.

Event Stalls

The `EventNode` has an additional option called 'Stall'. If the stall option for an `EventNode` is enabled it will prompt the Dialogue manager to stop traversing the dialogue and output a `DialogueInfo` with `dialogueType=DialogueType.Stall` and no other data. The `DialogueManager` will resume traversal the next time `AdvanceDialogue` is called. This is most often used for cases where you would like to pause the dialogue and later resume in response to some event. A common example of this might be to create a `WaitForSecond` event which will cause the `AdvanceDialogue` method to be called again after a delay.

Using an Event

The API of the `DSEvent` and `DSEvent<T>` are designed to be identical nearly identical to that of the `UnityEvent` and `UnityEvent<T>` classes respectively in order to improve familiarity. It uses the standard Pub/Sub architecture for events. `DSEvent`s that are attached to `EventNode`s will be invoked automatically, but they can also be invoked manually through code.

```
void DSEvent.Invoke()
void DSEvent.AddListener(UnityAction call)
void DSEvent.RemoveListener(UnityAction call)

void DSEvent<T>.Invoke(T value)
void DSEvent<T>.AddListener(UnityAction<T> call)
void DSEvent<T>.RemoveListener(UnityAction<T> call)
```

Notably, events exists as `ScriptableObject`s, which means they can be Invokes from multiple `DialogueManager`s without any way of distinguishing which was the source. For this purpose, an additional set of methods are defined which can add and remove listeners which only listen to events called from a specific `DialogueManager`.

```
void DSEvent.Invoke(DialogueManager manager)
void DSEvent.AddListener(DialogueManager manager, UnityAction call)
void DSEvent.RemoveListener(DialogueManager manager, UnityAction call)

void DSEvent<T>.Invoke(DialogueManager manager, T value)
void DSEvent<T>.AddListener(DialogueManager manager, UnityAction<T> call)
void DSEvent<T>.RemoveListener(DialogueManager manager, UnityAction<T> call)
```

Because these events exist as `ScriptableObjects` their data will persists between scenes. This makes is especially important to clean up their listeners properly, otherwise they may continue to grow in size. It is advised to call `RemoveAllListeners` when appropriate to reduce the risk of this occuring.

```
void DSEvent.RemoveAllListeners()
void DSEvent<T>.RemoveAllListeners()
```

Defining Event Types

While the default event types support a large number of use cases, you may need other types to be supported for specific use cases. While the generic type `DSEvent<T>` exists, `ScriptableObject`s do not support open generics. To make use of this class you must define a child class which closes the generic. You also must use the `[CreateAssetMenu]` attribute to make it available within the Create menu.

```
[CreateAssetMenu(fileName = "DSEvent", menuName = "Dialogue System/Events/Dialogue
Event (Custom)")]
class DSEventCustom : DSEvent<MyCustomClass> {}
```

Unless you'd like to implement your own functionality for an event type, no definitions are required inside the graph. The child class only needs to exists as to close the generic. Your custom DSEvent type will be automatically supported by the `EventNode` in dialogue graphs, so long as the generic type used is properly serilizeable.